REFINED QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
FINAL ROUND (Execution, RCA & Hard Evidence) **3**
1. Test Execution & Evidence (Manual) 3
2. Automated Testing Pipeline 5
2.1. Unit Testing (Core Logic & Functions) 5
2.2. Integration & API Testing (System Modules) 7
3. Defect Log & Fix Traceability (Root Cause Analysis) 10
4. Known Issues & Deferred Technical Debt 11
5. Live Demo / UAT Risks 12

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 — SettleOps AI
Team Repo URL	https://github.com/stanX19/SettleOps-AI
Project Board URL	https://github.com/stanX19/SettleOps-AI/issues
Live Deployment URL	https://settleops-ai.vercel.app/

Objective:

The primary objective is to ensure that SettleOps AI can reliably process insurance claim documents through a multi-agent AI pipeline, present structured decisions on a live dashboard, support full officer controllability (approve, decline, challenge), and maintain AI explainability across all reasoning stages, with the auditor agent's PEVR (Plan, Execute, Verify, Replan) cycle catching inconsistencies before human review.

FINAL ROUND (Execution, RCA & Hard Evidence)

1. Test Execution & Evidence (Manual)

Execute the tests drafted in **Part 1**. You must provide a link to visual proof for at least 3 critical tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
TC-01	**Happy Case (Entire Flow):** Entire Claim Flow	Verify that a claims officer can create a motor claim case, upload required documents, run AI claim analysis, review the blackboard, generate decision artifacts, and approve the	Claim case is created successfully, documents are processed, AI agents complete analysis, blackboard	Claim case is created successfully, documents are processed, AI agents complete analysis, blackboard is populated,	Pass	https://drive.google.com/file/d/18CWGEIAAXe_aRq2o5UZF_KU0jR_zKRyH/view?usp=sharing

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
		claim.	is populated, recommendation is generated, and decision PDF/audit artifact is produced.	recommendation is generated, and decision PDF/audit artifact is produced.		
TC-02	**Specific Case (Negative):** Missing Required Documents	Verify that the system blocks or warns the officer when a claim is submitted without required evidence such as police report, policy cover note, or damage photos.	System clearly identifies missing documents, prevents unsupported final decision, and does not crash.	System clearly identifies missing documents, prevents unsupported final decision, and does not crash.	Pass	https://drive.google.com/file/d/1ZgUaFTIDUp80O_zZx4_RQvhwUKafuLoC/view?usp=sharing
TC-03	**NFR (Performance):** Concurrent Claim API Load	Verify that the backend can handle multiple claim-related API requests at the same time, such as 3 concurrent case creation or upload requests.	Average response time remains below the agreed target, e.g.for lightweight API calls, with 0% server errors.	Average response time remains below the agreed target, e.g.for lightweight API calls, with 0% server errors.	Pass	https://drive.google.com/file/d/1PsOzTT8gvsyFHvGlcQQkwmKfpaa02N3n/view?usp=sharing
TC-AI-01	**AI Output Validation:** Claim Recommendation Accuracy	Verify that the AI agents analyze uploaded claim evidence and produce a grounded recommendation for policy, liability, damage, fraud, payout, and audit review.	AI output is consistent with uploaded documents, does not hallucinate missing facts, provides explainable reasoning, and calculates	AI output is consistent with uploaded documents, does not hallucinate missing facts, provides explainable reasoning, and calculates payout according to claim rules.	Pass	https://drive.google.com/file/d/14ajvEIKImDX8eZvf0b0KMc9WemloMpwz/view?usp=sharing

© UMHackathon 2026

Test Case ID	Test Type & Mapped Feature	Test Description	Expected Result	Actual Result	Status	Proof of Execution (Required for P0/Critical tests)
			payout according to claim rules.			

2. Automated Testing Pipeline

2.1. Unit Testing (Core Logic & Functions)

Unit testing is run in isolation at backend module level using mocked/fixture data to validate deterministic logic (citation normalization, PDF financial mapping, and rule validation) without requiring live frontend flows.

- **Test Runner Used**: `pytest` (Python backend)
- **Targeted Components**:
  - Citation validation and normalization logic
  - Financial field mapping from payout output to generated report payload
  - Agent-side adjuster range and payout checks
- **File(s) Covered**:
  - backend/tests/test_citation_validator.py
  - backend/tests/test_pdf_financials.py

© UMHackathon 2026

Unit Test ID	Function/Component Tested	Test Scenario	Expected Outcome	Actual Result	Status
UT-01	Citation text matching tolerance (`test_text_citation_tolerates_pdf_spacing_and_punctuation`)	Validate citation matching when PDF extraction introduces spacing/punctuation drift	Citation should still resolve to the intended source text with valid offsets	Passed in test run	PASS
UT-02	Label-value citation extraction (`test_label_value_citation_matches_split_policy_values`)	Validate split label/value policy fields can still be matched from extracted text	Validator should accept citation and map correct values	Passed in test run	PASS
UT-03	PDF payout financial mapping (`test_repair_approval_data_uses_payout_breakdown_financials`)	Validate report payload uses payout breakdown values for final totals	Financial fields in generated report payload should match payout results	Passed in test run	PASS
UT-04	Empty citation guard (`test_empty_citations_fail_hard`)	Validate defensive failure behavior when no citations are provided	Validator should fail fast instead of silently accepting incomplete evidence	Passed in test run	PASS

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
Problems   Output   Debug Console   Terminal   Ports

PS C:\Users\User\Documents\Visual Studio Code\SettleOps AI> cd backend
PS C:\Users\User\Documents\Visual Studio Code\SettleOps AI\backend> python -m pytest tests/test_citation_valida
tor.py tests/test_pdf_financials.py -q
C:\Users\User\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packag
es\Python311\site-packages\pytest_asyncio\plugin.py:247: PytestDeprecationWarning: The configuration option "as
yncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of py
test-asyncio will default the loop scope for asynchronous fixtures to function scope. Set the default fixture l
oop scope explicitly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "funct
ion", "class", "module", "package", "session"

  warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))
..............                                                                          [100%]
14 passed in 11.53s
PS C:\Users\User\Documents\Visual Studio Code\SettleOps AI\backend>
```

2.2. Integration & API Testing (System Modules)

This test focuses on module-to-module communication correctness (Frontend > Backend > Case runtime store / processing pipeline).

- **Test Tool Used**: pytest (backend integration suites), Python API harness scripts
- **Targeted Integrations**:
  - Frontend/API contract -> FastAPI case endpoints
  - FastAPI routes -> in-memory CaseStore and workflow orchestration
  - Workflow engine -> SSE event stream and snapshot topology
- **File(s) Covered**:
  - backend/tests/test_cases_routes.py
  - backend/tests/test_workflow.py
  - backend/tests/test_granular_sse_e2e.py
  - backend/tests/test_document_extraction.py

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test ID	Integration / Point Tested	Test Scenario	Expected Outcome	Actual Result	Status
IT-01	`POST /api/v1/cases` + route helpers	Run `python -m pytest tests/test_cases_routes.py -q` to validate case route behavior and case-backed text retrieval flow	API helper path executes without route/state errors	`1 passed in 28.04s`	PASS
IT-02	Workflow orchestration module integration	Run `python -m pytest tests/test_workflow.py -q` to validate multi-step workflow behavior across core modules	Workflow tests complete without orchestration exceptions	`4 passed in 18.89s`	PASS
IT-03	Granular SSE E2E harness (`/stream` + events)	Run `python -m pytest tests/test_granular_sse_e2e.py -q`	E2E suite should collect and verify SSE event contract	Test collection fails with `ModuleNotFoundError: No module named 'tests.test_client'`	FAIL

© UMHackathon 2026

Email: umhackathon@um.edu.my

IT-04	Document extraction integration fixtures	Run `python -m pytest tests/test_document_extraction.py -q` against fixture cases	Extraction method assertions should match runtime output contract	`5 failed` due to assertion mismatch (`expected gemini_vision`, got `rotating_llm_vision`)	FAIL

Execution Proof (Terminal):

```
python -m pytest tests/test_cases_routes.py -q
# 1 passed in 28.04s

python -m pytest tests/test_workflow.py -q
# 4 passed in 18.89s

python -m pytest tests/test_granular_sse_e2e.py -q
# ERROR: ModuleNotFoundError: No module named 'tests.test_client'

python -m pytest tests/test_document_extraction.py -q
# 5 failed (method contract mismatch: gemini_vision vs rotating_llm_vision)
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Defect Log & Fix Traceability (Root Cause Analysis)

Bug ID	Bug Description	Steps to Reproduce	Console/Error Log Snippet	Root Cause Analysis (Why did it break?)
DEF-01	Citation evidence mapping instability caused inconsistent decision trace readability	Run claim workflow, inspect citation anchors across reruns	Behavioral issue tracked in commit history	Citation path handling and payout fallback behavior were not robust across edge-case workflow states
DEF-02	Real-time workflow updates occasionally desynced in UI	Execute live workflow streaming with concurrent updates	Real-time update bottleneck symptoms addressed in fix commit	SSE resilience and sync handling needed hardening under frequent event transitions
DEF-03	Workflow retry edge-lane rendering issue in graph pane	Trigger retry-heavy workflow graph and inspect lane layout	Visual regression addressed in UI fix commit	Workflow pane edge constraint/styling did not properly handle retry lane routing

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. Known Issues & Deferred Technical Debt

Bug ID	Description & Impact (Bug or Technical Debt)	Reason for Deferral
DEF-04	**SSE E2E harness import path issue**: tests/test_granular_sse_e2e.py cannot be collected in current setup (ModuleNotFoundError: tests.test_client). Impact: automated full-stream regression check is blocked unless run-path/imports are normalized.	Core product flow works, so release focus remained on runtime behavior; test harness packaging fix deferred.
DEF-05	**Document extraction test contract drift**: integration test expects extraction method label gemini_vision while runtime now emits rotating_llm_vision. Impact: false-negative test failures despite successful extraction.	Runtime behavior was updated first; test assertions were not yet synchronized to new method naming contract.
DEF-06	**Volatile runtime storage in demo mode**: backend currently uses in-memory SQLite + in-memory CaseStore, so state is reset on backend restart. Impact: case continuity risk during long demos/restarts.	Accepted trade-off for hackathon velocity; persistent database migration deferred to next hardening cycle.

© UMHackathon 2026

Email: umhackathon@um.edu.my

5. Live Demo / UAT Risks

- **Risk 1:** Case data is demo-persistent only (in-memory state): Current backend uses in-memory storage for both DB and case runtime state. If backend restarts/redeploys, active case progress and draft workflow state can be lost.

  Operational instruction: Avoid restarting backend during judging; prepare one fallback case ID in advance.

- **Risk 2:** Workflow/artifact generation is asynchronous (do not spam actions). After starting workflow or generating outputs (especially report artifacts), results may not appear instantly; UI and SSE pipeline need time to complete.

  Operational instruction: Click action buttons once, then wait for status/progress before retrying or refreshing.

- **Risk 3:** Best experienced on desktop Chrome. Workflow UI is optimized for multi-pane desktop layout and dense evidence/blackboard interactions; small screens/browsers may render less consistently.

  Operational instruction: Run demo on latest Google Chrome, desktop resolution (recommended 1440px+ width).

© UMHackathon 2026

Email: umhackathon@um.edu.my